A. AKSHAYA
CSD
24NU1A4404

DATA ENGINEERING
ASSIGNMENT - 04

Q1. Explain the concept of a data lake and its importance in handling large-scale, diverse data. What are the key advantages and limitations of a data lake architecture?

DATA LAKE

A data lake is a centralized repository that stores large volumes of structured, semi-structured, and unstructured data in raw form. It follows a schema-on-read approach, meaning data structure is applied only when the data is accessed, not when it is stored.

* IMPORTANCE :

→ Scalability - Can store massive volumes of data using distributed storage systems.

→ Support for diverse data types - Can handle structured (tables), semi-structured (JSON, XML); and unstructured data (images, videos, text).

→ Decoupling Storage & Compute - Storage and processing are separated, allowing independent scaling.

* ADVANTANGE :

→ Flexible - No need for predefined schema

→ Centralized storage - Eliminates data silos

→ Supports analytics & ML workloads.

→ Uses low-cost storage system.

* LIMITATIONS :

→ Risk of data swamp if not managed properly.

→ Data quality issue due to lack of schema & cataloging.

→ Complex processing.

→ Write-only nature (hard to update / delete data)

Q2. ~~Briefly~~ Briefly explain different ways to ingest data into a data system - Direct Database Connection, CDC, APIs, Message Queues and Even-Streaming Platforms.

A) DATA INGESTION METHODS

Data ingestion is the process of collecting and transferring data from various sources into a data system for storage & analysis. There are multiple ways to ingest data, each suited to different use.

* Direct Database Connection - Data is ingested by directly connecting to source databases using queries.

- Uses tools like SQL to extract data
- Common for batch ingestion
- Simple to implement.

Limitation :

- Can impact source system performance
- Not suitable for real-time data

* Change Data Capture (CDC) - CDC captures only the changes (inserts, updates, deletes) made in the source database.

- Reads database logs instead of full tables
- Efficient and reduces data movement
- Supports near real-time ingestion
- Requires expertise to maintain

3. Discuss in detail about Batch ingestion - snapshot or differential extraction, File based export, ETL vs ELT and inserts, updates and batch size in ingestion.

A) BATCH INGESTION

Batch ingestion is the process of collecting and loading data in fixed intervals instead of real time. It processes data in large chunks, making it suitable for analytics and reporting.

* Snapshot vs Differential Extraction:

- Snapshot Extraction - Entire dataset is extracted every time, simple but inefficient for large data.
- Differential Extraction - Only changed data is extracted. Efficient but more complex.

* File-Based Export:

- Data is exported as files (CSV, JSON, Parquet) and then ingested.
- Simple & scalable.
- Limitation - High latency & file management issues.

* ETL vs ELT:

- ETL → Transform before loading (cleaner but slower)
- ELT → Load first, transform later (faster & scalable)

* Inserts, Updates, and Batch size:

- Inserts - Adding new records (simple)
- Updates - Modifying existing data (complex in batch systems)
- Batch size - Large → efficient but high latency
  small → faster updates but more overhead

Limitation :

- Complex to implement
- Requires support from database systems.

* API's (Application Programming Interfaces)

Data is ingested by calling API's provided by external systems.

- Common for SaaS applications & web services.
- Flexible & widely used.
- Supports real-time or batch ingestion

Limitation :

- Rate limits & latency issues
- Dependency on external systems.

* Messages Queues - Message queues act as intermediaries that store and deliver data between systems.

- Enables asynchronous communication.
- Decouples producers & consumers.
- Ensures reliability & fault tolerance

Limitation :

- Limited scalability compared to streaming platforms
- Not ideal for high-throughput real-time analysis

* Event-Streaming Platforms - These platforms handle continuous streams of data in real time.

- Examples include systems like Kafka
- Supports high-throughput, real-time processing
- Allows multiple consumers to read data simultaneously.

Limitations :

- Complex setup & management
- Requires expertise to maintain